Atelier JavaScript - Version prof

1. Le héros

Mission 01 - Nomme ton héros

Consigne : Choisis un nom court pour ton héros. Écris ta ligne avec `playerName` dans le fichier vide.

Réponse :

```
const playerName = "Ninja";
```

Nom libre de 2 à 16 caractères, autre que « player ». Modifier cette même ligne pour changer le nom.

Mission 02 - Deux noms, un seul héros

Consigne : Choisis un nouveau nom et vérifie. Change encore le nom sur la même ligne, puis vérifie.

Réponse : Modifier la même ligne avec deux noms différents, par exemple "Nova" puis "Luna", et vérifier après chaque nom.

QCM 01 - Lire une ligne

Question 1. Que contient playerName ?

```
const playerName = "Nova";
```

Réponse : A. Le texte Nova

Question 2. Pour changer le nom du héros, quelle partie modifies-tu ?

```
const playerName = "Nova";
```

Réponse : B. Le texte entre guillemets

Mission 03 - Choisis son départ

Consigne : Ajoute une ligne avec `startX`. Choisis un départ entre `24` et `70`.

Réponse :

```
const startX = 43;
```

Tout nombre de 24 à 70 est accepté.

Mission 04 - Deux points de départ

Consigne : Essaie un départ près de `30` et vérifie. Sur la même ligne, essaie près de `60` et vérifie.

Réponse : Sur la même ligne, tester `const startX = 30;` puis `const startX = 60;`, en vérifiant chaque fois. Deux positions séparées d'au moins 15 pixels sont acceptées.

Mission 05 - Gare ton héros

Consigne : Modifie `startX` pour partir entre `50` et `60`. Vérifie et regarde la position X sous le jeu.

Réponse : Par exemple `const startX = 55;`, puis vérifier. Toute valeur de 50 à 60 est acceptée.

QCM 02 - Comprendre la position

Question 1. On passe startX de 30 à 60. Où commence le héros ?

```
const startX = 60;
```

Réponse : B. Plus à droite

Question 2. Quelle valeur est bien un nombre ?

Réponse : C. 50

2. Les premiers pas

Mission 06 - Règle son moteur

Consigne : Ajoute le réglage `speed`. Choisis une vitesse de `3` à `5`.

Réponse :

```
const speed = 4;
```

Tout nombre de 3 à 5 est accepté.

QCM 03 - Un départ ou un pas ?

Question 1. Quelle donnée change la taille d'un pas ?

```
const startX = 40;
const speed = 5;
```

Réponse : A. speed

Question 2. Quelle modification ne change que le départ ?

Réponse : B. Changer startX

Mission 07 - Fais son premier pas

Consigne : Dans le bloc vert, complète le calcul de `player.x`. Garde sa position actuelle et ajoute un pas de `speed`.

Réponse :

```
function updateX(player, keys, speed) {
  player.x = player.x + speed;
}
```

Mission 08 - Change son allure

Consigne : Essaie `speed` à `3` et laisse le héros avancer. Passe près de `5`, vérifie et compare son allure.

Réponse : Tester `const speed = 3;` puis `const speed = 5;`. Vérifier et laisser avancer le héros d'au moins 12 pixels à chaque essai. La différence entre les vitesses doit être d'au moins 1.

QCM 04 - Prévoir le prochain pas

Question 1. Le héros est à X = 50 et speed vaut 4. Quelle est sa prochaine position ?

```
player.x = player.x + speed;
```

Réponse : B. 54

Question 2. Le jeu répète ce calcul deux fois depuis X = 50. Où arrive-t-on ?

```
player.x = player.x + 4;
```

Réponse : C. 58

Mission 09 - Donne-lui une commande

Consigne : Dans `updateX`, entoure ton calcul avec `if (keys.right) {` et `}`. Garde aussi les accolades de la fonction.

Réponse :

```
function updateX(player, keys, speed) {
  if (keys.right) {
    player.x = player.x + speed;
  }
}
```

Mission 10 - Avance, puis stop !

Consigne : Vérifie, puis avance avec → d'au moins `50` pixels. Relâche la touche pour montrer que le héros s'arrête.

Réponse : Après Vérifier, avancer de 50 pixels ou plus par rapport au départ de l'essai, puis relâcher la flèche droite. Le héros doit s'arrêter.

3. Les commandes

QCM 05 - Qui donne la commande ?

Question 1. La flèche droite est relâchée. Le calcul dans ce if se fait-il ?

```
if (keys.right) {
  player.x = player.x + speed;
}
```

Réponse : B. Non

Question 2. Que placent les accolades du if ?

Réponse : C. Les actions à faire si la condition est vraie

Mission 11 - Fais demi-tour

Consigne : Dans la même fonction, ajoute un deuxième `if` pour `keys.left`. Cette fois, enlève `speed` à `player.x`.

Réponse :

```
function updateX(player, keys, speed) {
  if (keys.right) {
    player.x = player.x + speed;
  }
  if (keys.left) {
    player.x = player.x - speed;
  }
}
```

Mission 12 - Aller... et retour !

Consigne : Vérifie, puis dépasse la position `X = 120` avec →. Reviens près de ton départ avec ←.

Réponse : Après Vérifier, atteindre X = 120 ou plus avec la flèche droite, puis revenir près de `startX` avec la flèche gauche. Une marge de 5 pixels est acceptée.

QCM 06 - Trouver le bon sens

Question 1. X vaut 80 et speed vaut 4. Où mène ce calcul ?

```
player.x = player.x - speed;
```

Réponse : A. 76, vers la gauche

Question 2. Le héros va à droite, puis à gauche avec le même pas. Quel est le résultat ?

Réponse : B. Il revient à sa position précédente

Mission 13 - Vise la plateforme

Consigne : Ajoute `jumpPower`, avec un nombre de 9 à 11. Vérifie, puis essaie un saut avec `Espace`.

Réponse :

```
const jumpPower = 10;
```

Tout nombre de 9 à 11 est accepté.

Mission 14 - Trouve ton saut

Consigne : Essaie une puissance près de 9, vérifie et saute. Essaie près de 11, vérifie et saute à nouveau.

Réponse : Tester `const jumpPower = 9;` puis `const jumpPower = 11;`. Vérifier et sauter à chaque essai. Deux puissances séparées d'au moins 1 sont acceptées.

Mission 15 - Pose-toi sur la plateforme

Consigne : Vérifie, puis avance vers la première plateforme. Saute et pose-toi dessus.

Réponse : Vérifier, sauter en avançant et atterrir sur la première plateforme. Par exemple : `const jumpPower = 10;`. Il faut se poser dessus, pas seulement la survoler.

4. Les sauts

Mission 16 - Teste la gravité

Consigne : Ajoute `gravity`, entre `0.35` et `0.6`. Teste deux valeurs et garde ton réglage préféré.

Réponse :

```
const gravity = 0.45;
```

Tout nombre de 0.35 à 0.6 est accepté, avec un point décimal.

Mission 17 - Un saut sur la Lune

Consigne : Essaie une gravité près de `0.35`, vérifie et saute. Essaie près de `0.6`, vérifie et saute à nouveau.

Réponse : Garder la même puissance, puis tester `const gravity = 0.35;` et `const gravity = 0.6;`. Vérifier et sauter à chaque essai. Deux gravités séparées d'au moins 0.1 sont acceptées.

Mission 18 - Atteins le sommet

Consigne : Vérifie tes réglages et rejoins la première plateforme. Saute ensuite sur la plateforme la plus haute.

Réponse : Vérifier, rejoindre la première plateforme puis sauter sur la plus haute. Un réglage possible : `const jumpPower = 10;` et `const gravity = 0.45;`.

QCM 07 - Prévoir un saut

Question 1. On garde la même gravité. Quel réglage fait monter plus haut ?

Réponse : A. jumpPower = 11 plutôt que 9

Question 2. On garde la même puissance. Avec quelle gravité retombe-t-on moins vite ?

Réponse : B. 0.35

Mission 19 - Envoie-le vers le haut

Consigne : Dans le nouveau bloc vert, change la réponse après `return`. Renvoie la valeur opposée de `jumpPower`.

Réponse :

```
function jumpSpeed(jumpPower) {
  return -jumpPower;
}
```

Mission 20 - Empêche le double saut

Consigne : Dans `canJump`, remplace `true` par l'information `player.onGround`. Teste `Espace` au sol, puis en plein saut.

Réponse :

```
function canJump(player) {
  return player.onGround;
}
```

5. Le score

Mission 21 - Essaie de tricher avec le saut

Consigne : Vérifie et fais un saut avec `Espace`. Pendant le saut, appuie encore sur `Espace`.

Réponse : Avec `return player.onGround;`, faire un premier saut, relâcher Espace puis appuyer à nouveau avant de retoucher le sol. Le second appui doit être refusé : un seul saut réel est effectué.

QCM 08 - Au sol ou dans les airs ?

Question 1. En plein saut, que vaut player.onGround ?

Réponse : B. false

Question 2. Quelle réponse doit donner canJump quand le héros est en l'air ?

```
return player.onGround;
```

Réponse : C. false, pour refuser le nouveau saut

Question 3. Avec jumpPower = 10, que renvoie ce calcul ?

```
return -jumpPower;
```

Réponse : A. -10

Mission 22 - Fais apparaître les fruits

Consigne : Ajoute `fruitPoints` avec `1`, `2` ou `3`. Vérifie, puis ramasse un fruit.

Réponse :

```
const fruitPoints = 1;
```

Valeurs acceptées : 1, 2 ou 3.

Mission 23 - Répare le compteur

Consigne : Le nouveau bloc oublie les anciens points : répare son `return`. Fais un calcul avec `score` et `points`.

Réponse :

```
function addPoints(score, points) {
  return score + points;
}
```

Mission 24 - Ramasse un duo

Consigne : Vérifie, puis ramasse le fruit près du départ. Ramasse un deuxième fruit et observe le nouveau score.

Réponse : Avec `return score + points;`, récolter au moins deux fruits après Vérifier. Le score doit garder les points des deux fruits : 2 points si chacun vaut 1, ou 4 si chacun vaut 2.

Mission 25 - Rends les fruits précieux

Consigne : Choisis une valeur de `fruitPoints`, vérifie et ramasse un fruit. Change la valeur, vérifie et ramasse encore un fruit.

Réponse : Tester par exemple `const fruitPoints = 1;` puis `const fruitPoints = 3;`. Vérifier et récolter un fruit à chaque essai. Deux valeurs entières différentes entre 1 et 3 sont acceptées.

QCM 09 - Garder les points gagnés

Question 1. Tu as 7 points et le fruit en ajoute 2. Quelle réponse faut-il garder ?

```
return score + points;
```

Réponse : C. 9

Question 2. Pourquoi return points est-il un bug pour le compteur ?

Réponse : A. Il remplace le score par la valeur du dernier fruit

6. Le trophée

Mission 26 - Trois fruits, six points

Consigne : Passe `fruitPoints` à 2 et vérifie. Ramasse au moins trois fruits pour atteindre 6 points.

Réponse : Écrire `const fruitPoints = 2;`, vérifier et récolter au moins trois fruits. Le score doit atteindre au moins 6 points.

Mission 27 - Place le trophée

Consigne : Ajoute `targetScore`, entre 3 et 5. Ramasse assez de fruits et rejoins le trophée.

Réponse :

```
const targetScore = 3;
```

Valeurs acceptées : 3, 4 ou 5.

Mission 28 - Gagne au bon moment

Consigne : Dans le nouveau bloc, corrige le signe de comparaison. Accepte un `score` égal ou supérieur à `target`.

Réponse :

```
function canWin(score, target) {
  return score >= target;
}
```

QCM 10 - Le score exact compte

Question 1. L'objectif vaut 3. Quel score cette règle accepte-t-elle ?

```
return score >= target;
```

Réponse : B. 3 et tous les scores plus grands

Question 2. Quelle erreur ferait le signe > à la place de >= ?

Réponse : A. Il refuserait le score exact

Mission 29 - Teste ton trophée

Consigne : Vérifie ton code et atteins ton objectif de points. Rejoins le trophée à droite du niveau.

Réponse : Après Vérifier, récolter assez de points pour atteindre ou dépasser `targetScore`, puis rejoindre le trophée à droite. La partie doit être gagnée.

Mission 30 - Attention au feu !

Consigne : Dans `isDangerous`, renvoie l'information `trap.active`. Observe le feu avant de traverser.

Réponse :

```
function isDangerous(trap) {
  return trap.active;
}
```

7. Le drapeau

Mission 31 - Traverse au bon moment

Consigne : Vérifie et rejoins le passage avec le feu. Traverse au sol quand le feu est éteint, sans te brûler.

Réponse : Après Vérifier, se placer au sol juste à gauche du feu, attendre qu'il soit éteint puis traverser en marchant, sans brûlure pendant l'essai. Rejouer après une brûlure.

Mission 32 - Active le drapeau

Consigne : Dans le bloc vert, compare `playerX` à `flagX`. Le drapeau doit s'activer quand le héros l'atteint ou le dépasse.

Réponse :

```
function canSave(playerX, flagX) {
  return playerX >= flagX;
}
```

Mission 33 - Éprouve ton drapeau

Consigne : Vérifie et dépasse le drapeau pour l'activer. Laisse le feu te toucher : observe où le héros revient.

Réponse : Après Vérifier, activer le drapeau puis toucher le feu allumé. Le héros doit revenir près du drapeau, autour de X = 195, au lieu du départ initial.

QCM 11 - Le rôle du drapeau

Question 1. Le héros est exactement à la position du drapeau. Cette règle l'active-t-elle ?

```
return playerX >= flagX;
```

Réponse : A. Oui

Question 2. Après activation, à quoi sert le drapeau quand le feu te touche ?

Réponse : C. Il donne un nouveau point de départ

Mission 34 - Dessine un chemin de fruits

Consigne : Ajoute `fruitGap`, entre 16 et 24. Compare deux écarts sur la première plateforme.

Réponse :

```
const fruitGap = 24;
```

Tout nombre de 16 à 24 est accepté.

Mission 35 - Dessine deux chemins

Consigne : Vérifie avec un écart près de 16. Change la même ligne vers 24 et compare la rangée.

Réponse : Sur la même ligne, tester `const fruitGap = 16;` puis `const fruitGap = 24;`, en vérifiant chaque fois. Deux écarts séparés d'au moins 4 sont acceptés.

8. Ta version du jeu

Mission 36 - Double la récompense

Consigne : Dans `fruitBonus`, calcule le double de `count`. Change la réponse après `return`.

Réponse :

```
function fruitBonus(count) {
  return count * 2;
}
```

Mission 37 - Encaisse ta prime

Consigne : Vérifie et ramasse au moins trois fruits. Gagne une partie et retrouve la prime à l'arrivée.

Réponse : Après Vérifier, récolter au moins trois fruits puis gagner. La prime doit valoir deux fois le nombre de fruits : 6 pour 3 fruits, 10 pour 5 fruits.

Mission 38 - Déverrouille l'arrivée

Consigne : Le bloc vérifie déjà le score. Ajoute `&& flag` pour exiger aussi le drapeau.

Réponse :

```
function canOpen(score, target, flag) {
  return score >= target && flag;
}
```

Mission 39 - Crée ta version du jeu

Consigne : Vérifie et fais un saut pour garder un premier essai. Change deux réglages parmi `speed`, `jumpPower`, `gravity`, puis vérifie et saute.

Réponse : Vérifier et sauter avec un premier réglage, puis changer au moins deux valeurs parmi `speed`, `jumpPower` et `gravity`, vérifier et sauter à nouveau. Exemple : 3 / 9 / 0.6, puis 4 / 10 / 0.45.

QCM 12 - Prêt à signer ton jeu ?

Question 1. Tu as assez de points, mais le drapeau est inactif. L'arrivée s'ouvre-t-elle ?

```
return score >= target && flag;
```

Réponse : B. Non, il manque le drapeau

Question 2. Tu as ramassé 5 fruits. Quelle prime renvoie ta fonction ?

```
return count * 2;
```

Réponse : C. 10

Question 3. Pour savoir quel réglage change le saut, quelle méthode utilises-tu ?

Réponse : B. Je change un réglage, puis je vérifie et je joue

Mission 40 - Signe ta victoire

Consigne : Invente ton message après `return`. Assemble ton texte, `playerName` et `score`, puis termine une partie.

Réponse :

```
function victoryMessage(playerName, score) {
  return "Bravo " + playerName + " ! Score : " + score;
}
```

Message libre contenant le nom et le score reçus. Terminer une partie pour valider la mission.